UNIT-II

Objective:

To familiarize how to employ regular expressions.

Syllabus:

Regular Expressions, Regular Sets, Identity Rules, Equivalence of two RE, Manipulations of REs, Finite Automata and Regular Expressions, Inter Conversion, Equivalence between FA and RE, Pumping Lemma of Regular Sets, Closure Properties of Regular Sets, Grammars, Classification of Grammars, Chomsky Hierarchy Theorem, Right and Left Linear Regular Grammars, Equivalence between RG and FA, Inter Conversion.

Learning Outcomes:

Students will be able to:

- understand the regular sets and how to represent the regular expressions.
- construct finite Automata for a given regular expression and viceversa.
- list closure properties of regular languages.
- understand the different applications of regular languages.
- understand Chomsky hierarchy of languages.
- understand and construct the regular grammar for the given regular language or regular expression.
- convert Regular Grammar into equivalent DFA and viceversa.

Learning Material

2.1 Regular set:

A language is a regular set (or just regular) if it is the set accepted by some finite automaton.

Example:

L= {0, 1, 10, 00, 01, 11, 000, 101, ...} is a regular set representing any no of 0's and any no of 1's.

2.2 Regular expression:

The languages accepted by finite automata are easily described by simple expressions called regular expressions.

Let Σ be an alphabet. The regular expressions over Σ and the sets that they denote are defined recursively as follows.

1) Ø is a regular expression and denotes the empty set.

2) ε is a regular expression and denotes the set { ε }.

3) For each a in Σ, a is a regular expression and denotes the set {a}.

4) If r and s are regular expressions denoting the languages R and S, respectively, then (r + s), (rs), and (r*) are regular expressions that denote the sets R U S, RS, and R*, respectively.

Some Examples on Regular expressions

1. Write regular expressions for each of the following languages over the alphabet {0, 1}.

   a) The set representing {00}.

      00

   b) The set representing all strings of 0's and 1's.

      (0+1)*

   c) The set of all strings representing with at least two consecutive 0's.

      (0 + 1)*00(0 + 1)*

   d) The set of all strings beginning with 1 and not having two consecutive 0' s.

      (1 + 10)*

   e) The set of all strings ending in 011.

      (0 + 1)*011

   f) The set of all strings representing any number of 0's followed by any number of 1's followed by any number of 2's.

      0*1*2*

   g) The set of all strings starting with 011.

      011 (0 + 1)*

2. Write regular expressions for each of the following languages over the alphabet {a, b}.

a) The set of all strings ending with either a or bb.

(a+b)* (a + bb)

b) The set of strings consisting of even number of a's followed by odd number of b's.

(aa)*(bb)*b

c) The set of strings representing even number of a's.

(b* a b* a b*) * + b*

2.3 Identity Rules Related to Regular Expressions

Given r, s and t are regular expressions, the following identities hold:

- $\emptyset^* = \varepsilon$
- $\varepsilon^* = \varepsilon$
- $r^+ = rr^* = r^*r$
- $r^*r^* = r^*$
- $(r^*)^* = r^*$
- $r + s = s + r$
- $(r + s) + t = r + (s + t)$
- $(rs)t = r(st)$
- $r(s + t) = rs + rt$
- $(r + s)t = rt + st$
- $(\varepsilon + r)^* = r^*$
- $(r + s)^* = (r^*s^*)^* = (r^* + s^*)^* = (r+s^*)^*$
- $r + \emptyset = \emptyset + r = r$
- $r\,\varepsilon = \varepsilon\, r = r$
- $\emptyset\, L = L\, \emptyset = \emptyset$
- $r + r = r$
- $\varepsilon + rr^* = \varepsilon + r^*r = r^*$

2.4 Equivalence of two RE

1. Equivalence of Regular Expressions means the situation where two different REs represent the same set of strings. In simpler words, if two REs generate the same language, they are considered equivalent.
2. Checking whether two REs are equivalent is a fundamental problem in the theory of computation. Let us see the methods for checking.

2.4.1 Methods for Proving Equivalence

3. There are primarily three methods for proving the equivalence of two REs −
    1. Set Generation
    2. Identities
    3. Machine Equivalence

1. **Set Generation:**

   This method involves generating the set of strings accepted by each RE and then comparing the two sets. If the sets are identical, the REs are equivalent. Let us understand this through an example − If we have RE1 = "1" and RE2 = "2," we can generate the set of strings accepted by each. Using RE1, the set would be {1}, and using RE2, the set would be {2}. Since these sets are different, we can conclude that RE1 and RE2 are not equivalent.

   On the other hand, if RE1 = "1" and RE2 = "1," the generated sets would both be {1}. Therefore, RE1 and RE2 are considered equivalent in this case.

2. **Identities:**

   This method uses a set of established identities that hold true for the regular expressions. These identities allow us to manipulate and simplify expressions. Ultimately checking whether the two expressions can be reduced to the same form or not. For instance, one identity states that **"ε + R* = R"** where Epsilon represents the empty string. If we have **RE1 = "ε + R"** and **RE2 = "R"** we can use this identity to simplify RE1 to **"R"** which matches RE2. Consequently, we can say that RE1 and RE2 are equivalent.

3. **Machine Equivalence**

   This method involves constructing finite automata (FA) for each RE and then checking whether the two FAs are equivalent or not. If the FAs accept the same set of strings, then the REs they represent are equivalent. The machine equivalence method is typically used for more complex REs where set generation and identity application become problematic.

   Examples to Prove Equivalence with Identities

Example 1: Let us consider an example to explain the use of identities for proving RE equivalence. Suppose we want to prove that

$-(1+(00)*1)+(1+(00)*1)(0+(10)*1)*(0+(10)*1)=0*1(0+(10)*1)*$

Solution:

We can start by simplifying the left-hand side (LHS) of the equation using the identities −

The LHS is $(1 + (00)^* 1) + (1 + (00)^* 1) (0 + (10)^* 1)^* (0 + (10)^* 1)$

Let us take $(1 + 00^* 1)$ as a common factor, so

(1+ (00)*1)(ε+(0+(10)*1)*(0+(10)*1))

(ε+R*R),where R=(0+(10)*1)

As we know,

(ε+R*R)=(ε+RR*)=R*

(1+(00)*1)((0+(10)*1)*)

out of this consider,

(1+(00)*1)(0+(10)*1)*

Taking 1 as a common factor, we can get

(ε+(00)*)1(0+(10)*1)*

Applying $\varepsilon + (00)^* = 0^*$, we get

0*1(0+(10)*1)*

Which is the R.H.S.

Example 2: Prove that $(0^* 1^*)^* = (0 + 1)^*$

Solution:

The LHS is $(0^* 1^*)^*$

$(0*1*)*=\{\varepsilon,0,00,1,11,111,01,10,...\}$

This is any combination of 0's, any combination of 1's, any combination of 0 and 1, ε

On the RHS, $(0 + 1)^*$

$(0+1)*=\{\varepsilon,0,00,1,11,111,01,10,...\}$

This is also any possible combination of 0s and 1s including null.

So, the sets are similar and these are equivalent.

2.5 Manipulations of REs

If r1 and r2 are regular expressions, then then

1. r1* is a RE
2. r1+r2 is a RE
3. r1.r2 is a RE

- **Union:** If R1 and R2 are regular expressions, then R1 | R2 (also written as R1 U R2 or R1 + R2) is also a regular expression.

$$L(R1|R2) = L(R1) \cup L(R2).$$

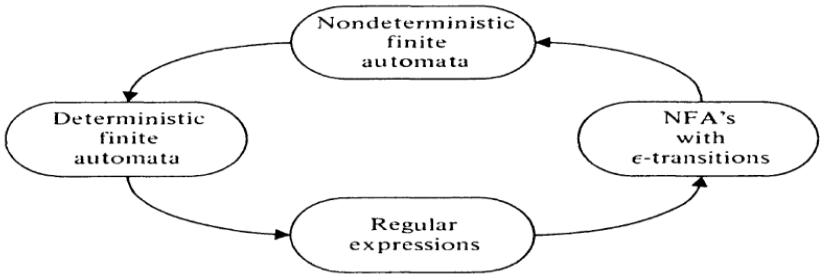
- **Concatenation:** If R1 and R2 are regular expressions, then R1R2 (also written as R1.R2) is also a regular expression.

$$L(R1R2) = L(R1) \text{ concatenated with } L(R2).$$

- **Kleene closure:** If R1 is a regular expression, then R1* (the Kleene closure of R1) is also a regular expression.

$$L(R1^*) = \text{epsilon} \cup L(R1) \cup L(R1R1) \cup L(R1R1R1) \cup \ldots$$

2.6 Construction of Finite automata for a given regular expression



Equivalence of Finite Automata and Regular Expressions

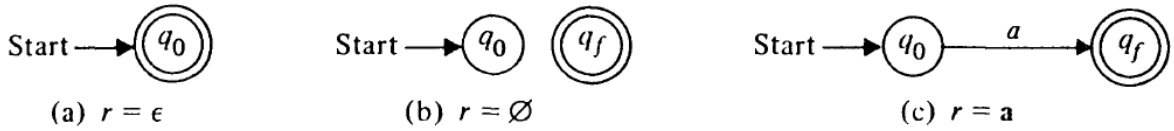
- The languages accepted by finite automata are precisely the languages denoted by regular expressions.

- For every regular expression there is an equivalent NFA with ε -transitions.
- For every DFA there is a regular expression denoting its language.

Let r be a regular expression. Then there exists an NFA with ε -transitions that accept L(r).

Zero operators:

The expression r must be ε, Ø, or a for some a in Σ. The NFA's for zero operators are



(a) $r = \epsilon$ (b) $r = \emptyset$ (c) $r = \mathbf{a}$

One or more operators:

Let r have i operators. There are three cases depending on the form of r.

Case 1: Union (r = rl + r2.)

There are NFA's M1 = **(Q1, ∑1, δ1, q1, {f1})** and M2=**(Q2, ∑2, δ2, q2, {f2})** with L(M1) = L(r1) and L(M2) = L(r2).

Construct

M = (Q1 ∪ Q2 ∪ {q0, f0}, **∑1** ∪ **∑2**, **δ, q0, {f0})** where **δ is defined by**

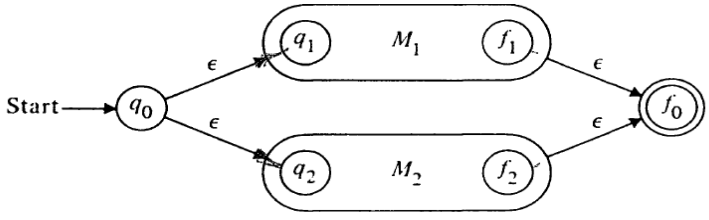
i) **δ (q0,** ε) = {q1,q2}

ii) **δ (q,** a) = **δ**1(q ,a) for q in Q1-{f1} and a in **∑1** ∪ { ε }

iii) **δ (q,** a) = **δ**2(q ,a) for q in Q2-{f2} and a in **∑2** ∪ { ε }

iv) **δ (f1,** ε) = **δ**1(f2, ε) = { f0 }

L(M) = L(M1) ∪ **L(M2)**



Case 2: Concatenation (r = r1 r2).

Let M1 and M2 be as in Case 1 and construct M = (Q1 ∪ Q2, **∑1 ∪ ∑2, δ, q1, {f2})**

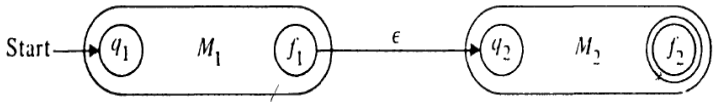
where **δ is defined by**

i) **δ (q,** a) = **δ**1(q ,a) for q in Q1-{f1} and a in **∑1** ∪ { ε }

ii) **δ (f1,** ε) = {q2}

iii) **δ (q,** a) = **δ**2(q ,a) for q in Q2 and a in **∑2** ∪ { ε }

L(M) ={xy| x is in L(M1) and y is in L(M2)} and L(M) = L(M1)L(M2)



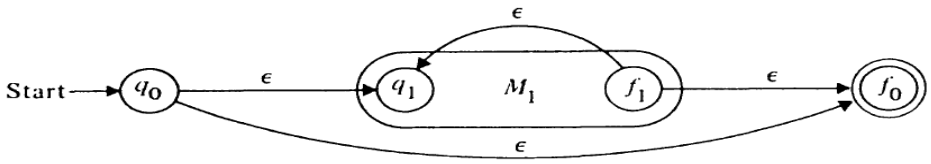
<u>Case 3:</u> Closure (r = r1*)

Let M1 = **(Q1, ∑1, δ1, q1, {f1}) and L(M1) = r1.**

Construct M = (Q1 ∪ {q0,f0}, **∑1** , **δ, q0, {f0}),** where **δ is defined by**

i) **δ (q0,** ε) = **δ (f1,** ε) = {q1,f0}

ii) **δ (q,** a) = **δ**1(q ,a) for q in Q1-{f1} and a in **∑1** ∪ { ε }

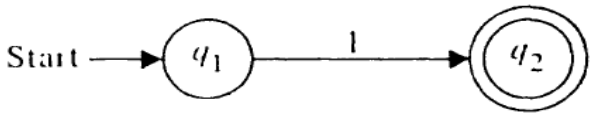


Example:

1. Construct an NFA for the regular expression 01^*+1

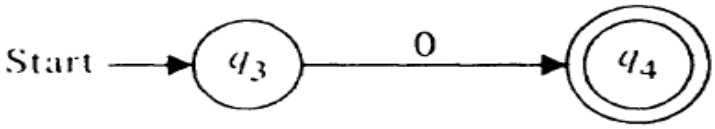
Regular expression is of the form r1 + r2, where r1 =01* and r2 = 1.

The automaton for r_2 is



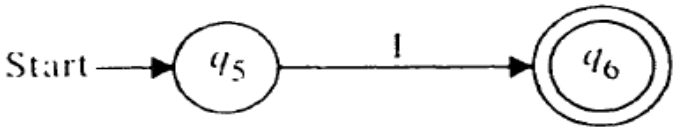
Express r_1 as r_3 and r_4, where $r_3=0$ and $r_4= 1^*$

The automaton for r_3 is

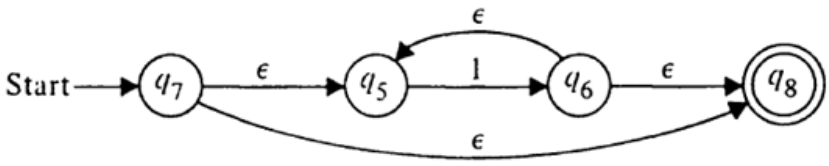


r_4 is r_5^* where $r_5=1$

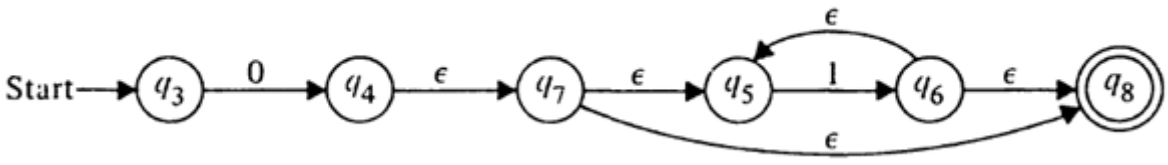
The NFA for r_5 is



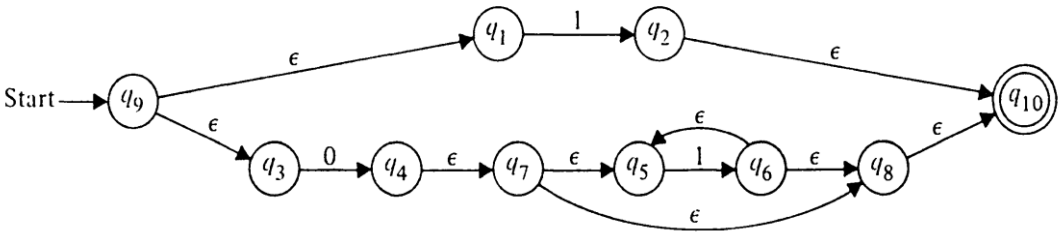
To construct an NFA for r4 = r_5^* use the construction of closure. The resulting NFA for r4 is



Then, for r1 = r3 r4 use the construction of concatenation.



Finally, use the construction of union to find the NFA for r = r1 + r2



2.7 Inter Conversion Construction of regular expressions for the given finite Automata

Arden's Theorem

Let P and Q be two regular expressions over Σ,and if P does not contain epsilon ,then R=Q+RP has a unique solution R=QP*.

Procedure

Assume the given finite automata should not contain any epsilons.

Step 1: Find the reachability for each and every state in given Finite automata.

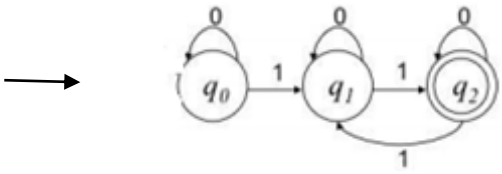
Reachability of a state is the set of states whose edges enter into that state.

Step 2: For the initial state of finite automata ,add epsilon to the reachability equation.

Step 3: Solve the equations by using Arden's Theorem.

Step 4: Substitute the results of each state equation into the final state equation,to get the regular expression for the given DFA.

Example:



The given Finite Automata is not having any ε's(***epsilons).***

Step 1: Find the reachability for each and every state in given Finite automata.

Reachability of a state is the set of states whose edges enter into that state.

$q_0 = q_0\,0$ ——— **1**

$q_1 = q_0\,1 + q_1 0 + q_2\,1$ ——— **2**

$q_2 = q_1\ 1 + q_2 0$ ——— **3**

Step 2: For the initial state of finite automata, add epsilon to the reachability equation.

$q_0 = q_0\ 0 + \varepsilon$

Step 3: Solve the equations by using Arden's Theorem.

After applying arden's theorem for equation 3

$q_2 = q_1\ 10^*$ ——— **4**

Substitute equation 4 in equation 2

$q_1 = q_0\ 1 + q_1 0 + q_1\ 10^*$

$q_1 = q_0\ 1 + q_1(0+10^*)$ ——— **5**

Apply arden's theorem on equation 5

$q_1 = q_0 1\ (0+10^*)^*$ ——— **6**

Apply arden's theorem on equation 1

$q_0 = q_0\ 0 + \varepsilon$

$q_0 = \varepsilon\ 0^*$ ——— **7**

Substitute equation 7 in equation 6

$q_1 = \varepsilon\ 0^*\ 1\ (0+10^*)^*$ ——— **8**

Step 4: Substitute the results of each state equation into the final state equation, to get the regular expression for the given DFA.

$q_2 = \varepsilon\ 0^*\ 1\ (0+10^*)^*\ 10^*$

Therefore, the regular expression for the given DFA is **0* 1 (0+10*)* 10*.**

2.8 Pumping Lemma for Regular Sets:

- Pumping lemma, which is a powerful tool for proving certain languages non-regular.
- It is also useful in the development of algorithms to answer certain questions concerning finite automata, such as whether the language accepted by a given FA is finite or infinite.

Lemma

Let L be a regular set. Then there is a constant n such that if z is any word in L, and $|z| > n$, we may write z=uvw in such a way that $|uv| \leq n$, $v \geq 1$, and for all $i > 0$, uv^iw is in L. Furthermore, n is no greater than the number of states of the smallest FA accepting L.

Example:

The set $L = \{0^{i^2} \mid i \text{ is an integer}, i \geq 1\}$, which consists of all strings of 0's whose length is a perfect square, is not regular.

Assume L is regular and let n be the integer in the pumping lemma.

Let $z = 0^{n^2}$.

By the pumping lemma, 0^{n^2}may be written as uvw, where $1 \leq |v| \leq n$ and uv^iw is in L for all i.

Let i = 2, $n^2 < |uv^2w| < n^2 + n < (n+1)^2$.

That is, the length of uv^2w lies properly between n^2 and $(n+1)^2$, and is thus not a perfect square.

Thus uv^2w is not in L, a contradiction.

We conclude that L is not regular.

2.9 Closure Properties of Regular Sets:

- The regular sets are closed under union, concatenation, and Kleene closure.
- The class of regular sets is closed under complementation. That is, if L is a regular set and $L \subseteq \sum^*$, then $\sum^* - L$ is a regular set.
- The regular sets are closed under intersection.
- The class of regular sets is closed under substitution.
- The class of regular sets is closed under homomorphism and inverse homomorphism.
- The class of regular sets is closed under quotient with arbitrary sets.

2.10 Grammars:

Grammar is a formal system that defines a set of rules for generating valid strings within a language. It serves as a blueprint for constructing syntactically correct sentences or meaningful sequences in a formal language.

Grammar is basically composed of two basic elements:

- **Terminal Symbols:** Terminal symbols are those that are the components of the sentences generated using grammar and are represented using small case letters like a, b, c, etc.
- **Non-Terminal Symbols:** Non-terminal symbols are those symbols that take part in the generation of the sentence but are not the component of the sentence. Non-Terminal Symbols are also called Auxiliary Symbols and Variables. These symbols are represented using a capital letters like A, B, C, etc.

Representation of Grammar

Any Grammar can be represented by 4 tuples - <V, T, P, S>

V - Finite Non-Empty Set of Non-Terminal Symbols.

T - Finite Set of Terminal Symbols.

P - Finite Non-Empty Set of Production Rules.

S - Start Symbol (Symbol from where we start producing our sentences or strings).

Production Rules

A production or production rule in computer science is a rewrite rule specifying a symbol substitution that can be recursively performed to generate new symbol sequences. It is of the form α-> β where α is a Non-Terminal Symbol which can be replaced by β which is a string of Terminal Symbols or Non-Terminal Symbols.

Example 1

Consider Grammar G1 = <V, T, P, S>

T = {a,b} #Set of terminal symbols

P = { A -> Aa, A -> Ab, A -> a ,A -> b, A -> $\boldsymbol{\varepsilon}$} #Set of all production rules

S = {A} #Start Symbol

As the start symbol S is equivalent to A then we can produce Aa, Ab, a, b, $\boldsymbol{\varepsilon}$ strings. These strings can further produce strings where A can be replaced by the strings mentioned in the production rules. Hence this grammar can be used to produce strings of the form (a+b)*.

Derivation of Strings

A->a #using production rule 3

OR

A->Aa #using production rule 1

Aa->ba #using production rule 4

OR

A->Aa #using production rule 1

Aa->AAa #using production rule 1

AAa->bAa #using production rule 4

bAa->ba #using production rule 5

Example 2

Consider Grammar G2 = <N, T, P, S>

V = {A} #Set of non-terminals Symbols

T = {a} #Set of terminal symbols

P = {A -> Aa, A -> AAa, A -> a, A -> ε} #Set of all production rules

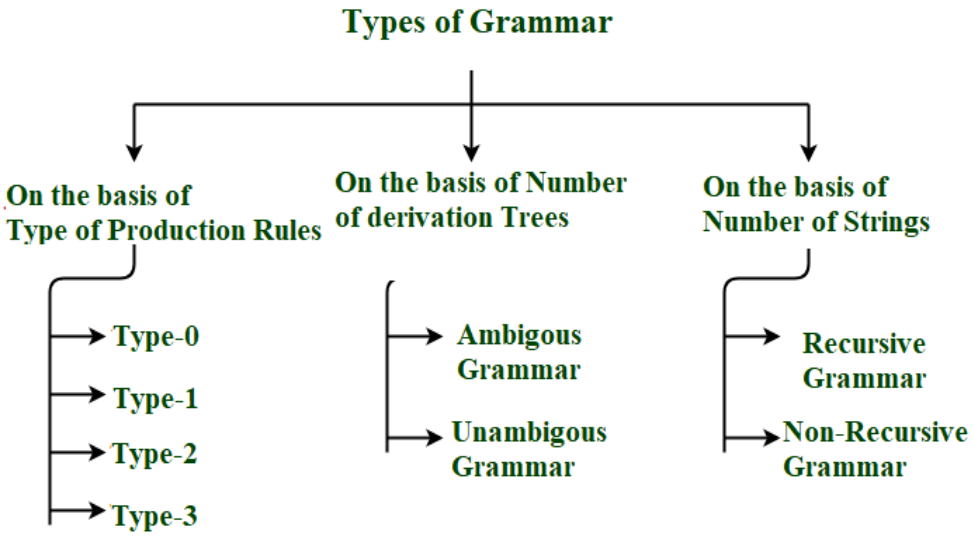
S = {A} #Start Symbol

As the start symbol is S then we can produce Aa, AAa, a, which can further produce strings where A can be replaced by the Strings mentioned in the production rules and hence this grammar can be used to produce strings of form (a)*.

2.11 Classification of Grammars:

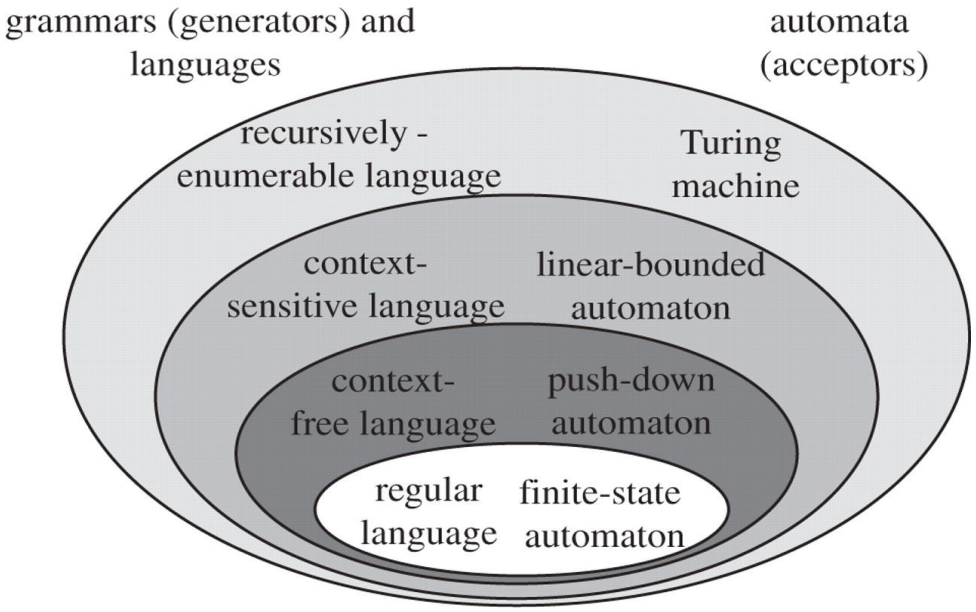
There are several types of Grammar. We classify them on the basis mentioned below.

- **Type of Production Rules**: The form and complexity of the production rules define the grammar type, such as context-free, context-sensitive, regular, or unrestricted grammars.
- **Number of Derivation Trees:** The number of ways a string can be derived from the grammar. Ambiguous grammars have multiple derivation trees for the same string.
- **Number of Strings:** The size and nature of the language generated by the grammar.



2.12 Chomsky Hierarchy Theorem:

The four classes of languages are often called the Chomsky hierarchy, after Noam Chomsky, who defined these classes as potential models of natural languages.



the traditional Chomsky hierarchy

Chomsky classifies the grammar into four types:

Grammar	**Languages**	**Automaton**	**Production rules**
Type 0	Recursively Enumerable/ Phrase Structured Languages	Turing Machines (TM)	$\alpha \rightarrow \beta$
Type 1	Context-Sensitive Languages	Linear-bound Automata (LBA)	$\alpha \rightarrow \beta$ $\lvert \alpha \rvert <= \lvert \beta \rvert$
Type 2	Context-Free Languages	Push-Down Automata (PDA)	$A \rightarrow \alpha$
Type 3	Regular Languages	Finite-State Automata (FSM)	$A \rightarrow w$ $A \rightarrow wB$ $A \rightarrow Bw$

2.13 Right and Left Linear Regular Grammars:

Regular Grammar:

A right- or left-linear grammar is called a regular grammar.

Right-Linear Grammar:

If all productions of a CFG are of the form A → wB or A → w, where A and B are variables and w is a (possibly empty) string of terminals, then we say the grammar is right-linear.

Left-Linear Grammar:

If all productions are of the form A → Bw or A → w, we call it left-linear.

Example:

Represent the language 0(10)* by the right-linear grammar.

The language generated by the given Regular Expression is

L = {0, 010, 01010, 0101010, ...}

Right-Linear Grammar:

S→0A

A→10A | ε

Example:

Represent the language 0(10)* by the left-linear grammar.

The language generated by the given Regular Expression is

L = {0, 010, 01010, 0101010, ...}

Left-Linear Grammar:

S→S10 | 0

2.14 Equivalence between RG and FA:

A language is regular if and only if it has a left-linear grammar and if and only if it has a right-linear grammar.

Construction of a Regular Grammar for a given DFA:

Let M = ({q0, q1... qn} **, ∑, δ, q_0, F)**. We construct G as G = ({A0, A1, ..., An},**∑, P, A0)** where P is defined by the following rules:

(i) Ai → aAj is included in P if **δ**(qi, a) = qj ∉ F.

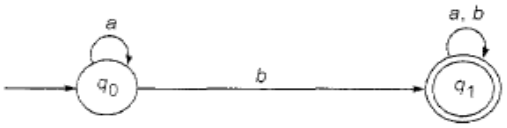
(ii) Ai → aAj and Ai → a are included in P if **δ**(qi, a) = qj ∈ F.

Note: We can construct only right linear grammar for the given DFA.

If we want to construct **left linear grammar** for the given DFA, reverse the edges of the given DFA and interchange initial and final states.

Example:

1. ***Construct regular grammar (right linear grammar) for the given DFA.***



Given M= ({q0,q1}, {a,b}, **δ, q_0, {q1})**

Construct G= ({A0,A1}, {a,b} ,P, A0) where *P* is given by

(i) Ai → aAj is included in P if δ(qi, a) = qj ∉ F.

δ(q0, a) = q0 ∉ F ⇒ A0→aA0

(ii) Ai → aAj and Ai → a are included in P if δ(qi, a) = qj ∈ F.

δ(q0, b) = q1∈ F ⇒ A0→bA1 and A0→b

δ(q1, a) = q1∈ F ⇒ A1→aA1 and A1→a

δ(q1, b) = q1∈ F ⇒ A1→bA1 and A1→b

∴ P is given by

A0→aA0, A0→bA1, A0→b

A1→aA1, A1→a, A1→bA1, A1→b

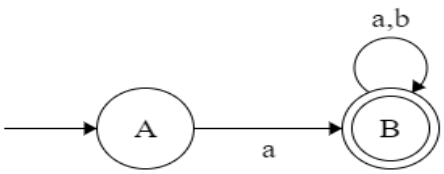
<u>Steps to convert Finite Automata to Left Linear Grammar</u>:

Step 1: Reverse all the edges of the given automata and interchange initial state and final states.

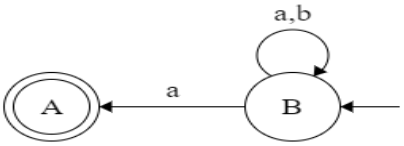
Step 2: Represent the productions using Left Linear Grammar.

Example:

2. ***Construct left linear grammar for the given DFA.***



Step 1: Reverse all the edges of the given automata and interchange initial state and final states.



Step 2: Represent the productions using Left Linear Grammar.

B→Ba B→Aa

B→Bb B→a

2.15 Inter Conversion: Construction of a DFA for a given Regular Grammar

Let G = **({A0, A1, ..., An},∑, P, A0).** We construct a DFA M whose

(i) states correspond to variables.

(ii) initial state corresponds to A0.

(iii) transitions in M correspond to productions in P. As the last production applied in any derivation is of the form Ai→ a, the corresponding transition terminates at a new state, and this is the unique final state.

We define M as **({q0, q1... qn, qf} , ∑, δ, q_o, {qf})** where **δ** is defined as follows:

(i) Each production Ai→aAj induces a transition from qi to qj with label a,

(ii) Each production Ak →a induces a transition from qk to qf with label a.

Example:

1. G= ({A0, A1}, {a,b} ,P, A0) where P consists of A0→aA1, A1→bA1, A1→a, A1→bA0. Construct a DFA M accepting L(G).

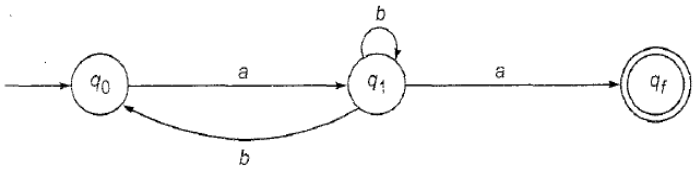
A0→aA1 induces a transition from q0 to q1 with label a.

A1→ bA1 induces a transition from q1 to q1 with label b.

A1→bA0 induces a transition from q1 to q0 with label b.

A1→a induces a transition from q1 to qf with label a.

M = **({q0, q1, qf} , ∑, δ, q_o, {qf}),** where q0 and qf correspond to A0 and A1 respectively and qf is the new final state introduced.

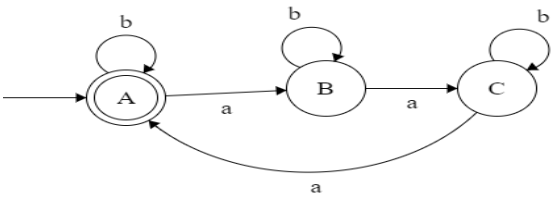


2. Construct Finite Automata for the grammar which consists of the productions

A → aB | bA | b

B → aC | bB

C → aA | bC | a



Assignment-Cum-Tutorial Questions

A. Questions testing the understanding / remembering level of students

I) Objective Questions

1. The languages accepted by finite automata are easily described by simple expressions called_____.
2. A language is a _____ if it is the set accepted by some finite automaton.
3. What is the solution for equation R=Q+RP (if P and Q are RE and P does not contain ^)? [ ]

   (a) $R=QP^*$ (b) R=QP (c) $R=PQ^*$ (d) $R=P^*Q^*$

4. $\emptyset + R =$_____.
5. $\emptyset^* =$_____.
6. $\varepsilon^*=$_____.
7. $\varepsilon + r\, r^*=r^*$ [True / False]
8. Pumping lemma is generally used for proving [ ]

   (a) a given grammar is regular (b) a given grammar is not regular

   (c) whether two given regular expressions are equivalent (d) none of the above

9. The logic of pumping lemma is a good example of [ ]

   (a) The Pigeon-hole principle (b) The divide and conquer technique

(c) Recursion (d) Iteration

10. Regular sets are closed under [ ]

(a) Union (b) concatenation (c) Kleene closure (d) All of the above

11. Match the following

1. Context Free Language	a. Turing Machine	[ ]
2. Recursively Enumerable	b. Finite Automata	[ ]
3. Regular Language	c. Linear Bounded Automata	[ ]
4. Context Sensitive Language	d. Push Down Automata	[ ]

II) Descriptive questions

1. Define regular set and regular expression.
2. State Arden's Theorem.
3. Explain the procedure to convert DFA to regular expression with an example.
4. List the closure properties of Regular Languages.
5. Explain pumping lemma for regular languages with an example.
6. Explain Chomsky hierarchy of languages.

B. Question testing the ability of students in applying the concepts.

I) Multiple Choice Questions:

1. a + b denotes the set_____. [ ]

a. {a, b} b. {ab} c. { a } d. {b}

2. The set of all strings of {0, 1} having exactly two 0's is [ ]

a) $1^*01^*01^*$ b) $\{(0+1)^*\}$ c)$\{11+0\}^*$ d)$\{00+11\}^*$

3. The regular expression to represent all strings with length atmost 2 over {a,b} is_____.

a) ε b) ε+(a+b)+(a+b).(a+b) c) (a+b) d) (a+b).(a+b)

4. Which of the following regular expressions denote 0 or more instances of a or b? [ ]

a) (a+b) b) (a+b)* c) (ab)* d) a*+b

5. Which of the following regular expressions denote 1 or more instances of a or b? [ ]

a) $(a+b)^+$ b) (a+b)* c) (ab)* d) a*+b

6. Which one of the following languages over the alphabet {0 ,1} is described by the regular expression: (0+1)*0(0+1)*0(0+1)*? [ ]

(a) The set of all strings containing the substring 00.

(b) The set of all strings containing atmost two 0's.

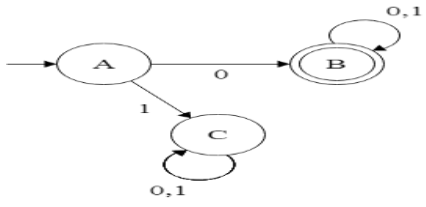
(c) The set of all strings containing atleast two 0's.

(d) The set of all strings that begin and end with either 0 or 1.

7. Consider the languages L1 = Ɛ and L2 = {0}. Which one of the following represents L1 $L2^{*}$ + L1* [ ]

(A) { Ɛ } (B) Ø (C) 0* (D) { Ɛ ,0}

8. What is the regular expression for the given DFA?

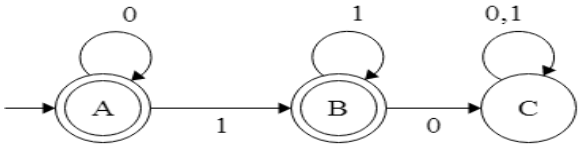


a) (0+1)* b) 0(0+1)* c) 0 d) (0+1)*0

9. Which of the following languages are not regular? [ ]

i) L= a^n | n>=1 ii) L=$a^n b^m$ | n,m>=1 iii) $a^n b^n$ | n>=1 iv) a^{2n} | n>=0

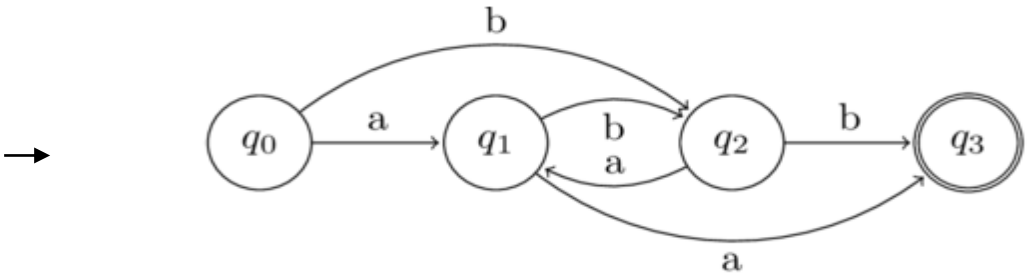
10. What is the regular expression for the given DFA? [ ]



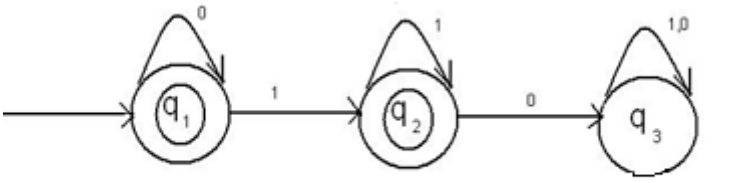
a) $0*1^{+}$ b) 0*1* c) 1*0* d)$1*0^{+}$

II) Problems

1. Write the regular expression for all strings ending in 1101 over the alphabet {0, 1}.
2. Write the regular expression for all strings beginning with 101 over the alphabet {0, 1}.
3. Write the regular expression for all strings which contains 01 over the alphabet {0, 1}.
4. Write regular expression to represent the set of strings of 0's and 1's whose tenth symbol from the right end is 1.
5. Design a ε-NFA for the regular expression a*bc | ab* |c*.
6. Construct Finite automata for the regular expression 1(01+10)*00.
7. Construct regular expression for the following DFA.



8. Construct regular expression for the given DFA.



C. Questions testing the analyzing /evaluating ability of students.

1. Analyze the language represented by the regular expression (1+01+001)*(ε +0+00)

D. GATE/NET/SLET

1. Consider the language L=(111+11111)*.The minimum number of states in any DFA accepting this language is: [ ] **GATE 2006**

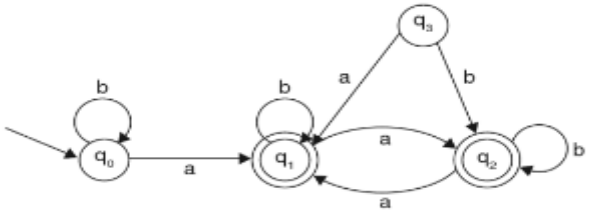
   (a)3 (b) 5 (c) 8 (d) 9

2. Consider the languages L1 = Ø and L2 = {a}. Which one of the following represents L1 L2 * U L1* [ ] **GATE 2013**

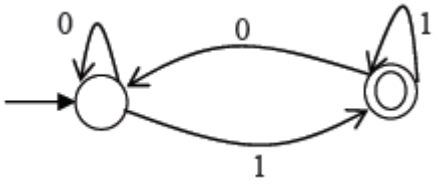
   (A) { Ɛ } (B) Ø (C) a* (D) { Ɛ ,a}

3. The language accepted by this automaton is given by the regular expression [ ]

   a) b*ab*ab*ab* b) (a+b)* c) b*a(a+b)* d) b*ab*ab* **GATE 2007**



4. Which of the regular expressions given below represent the following DFA? [ ]



GATE 2014 Set-1

I) 0*1(1+00*1)*
II) 0*1*1+11*0*1
III) (0+1)*1

(A) I and II only (B) I and III only (C) II and III only (D) I , II and III only

5. The number of states in the minimum sized DFA that accepts the language defined by the Regular expression (0+1)*(0+1)(0+1)* is_____. **GATE 2016 Set-B**

6. Let L = {w ∈ (0 + 1)*|w has even number of 1s}, i.e. L is the set of all bit strings with even number of 1s. Which one of the regular expressions below represents L?
(A) (0 *10 *1) * (B) 0 * (10 *10 *) * **GATE 2010**
(C) 0 * (10 *1*) * 0 * (D) 0 *1(10 *1) *10 * [ ]